

Upgrade Plan: Guarding the Recommendation Pipeline

Project: SVD / ALS Factorization with Fake Review Filtering Current state: DistilBERT, Naive Bayes, SVM and Decision Tree classifiers trained on a labeled fake review CSV. Feature and embedding extraction scripts exist. SVD and ALS are still to be built.

0. Priorities and Timeline

The core claim of the project is that removing fake reviews improves recommendations. Everything below either makes that claim provable or makes the pipeline more rigorous. Build in this order.

1. Phase 1, foundation: fix the data gap (Plan 1), baseline classifiers with proper evaluation (Plan 5, stages A and B), cleaned rating matrix, plain SVD and ALS.
2. Phase 2, core contribution: attack simulation (Plan 2), soft weighting (Plan 3), ALS from scratch (Plan 4).
3. Phase 3, depth: evaluation rigor (Plan 8), trust score (Plan 9), explainability (Plan 7).
4. Phase 4, extras if time allows: robustness tests (Plan 6), graph detection (Plan 10), cold start hybrid (Plan 11).
5. Phase 5, delivery: demo and packaging (Plan 12), report updates (Plan 13).

Suggested split for four members: one owns data and detector, one owns the recommenders, one owns the attack lab and evaluation, one owns the demo, explainability and report. Everyone reviews each other's code before results are frozen.

Plan 1. Fix the Data Foundation

Goal. Make the detector and the recommender work on the same kind of data, and be honest about what each dataset can support.

The problem. The labeled fake review CSV in the repo appears to be a text dataset. If it is the commonly used one with computer generated versus original labels, it has a category, a rating, a label and the text, but no reviewer ID and no timestamp. The Amazon Electronics ratings file used in the Kaggle reference notebook has user, item, rating and timestamp, but no review text. Your proposal needs text, user ID and timestamp together to compute behavior features. Also note that a computer generated versus original label teaches the model to spot machine written text, which is not the same as paid human fakes. State this clearly as a limitation.

Steps.

1. Get a version of Amazon Reviews that includes review text, reviewer ID, product ID, rating and timestamp for Electronics. The McAuley lab releases (the 2018 and 2023 versions) have these. Check the current download page and file sizes before committing.
2. Apply the k core filter: keep users and items with at least k ratings (try k = 5, report the resulting user, item and rating counts and the matrix density).
3. Use the labeled fake dataset only to train and validate the text classifier. Apply the trained classifier to the Amazon reviews to get a text fake probability for every review.
4. Compute behavior features on the Amazon data (listed below).
5. Because the Amazon data has no fake labels, validate the full pipeline with injected attacks (Plan 2), where ground truth is known.

Behavior features to compute with pandas groupby and rolling windows.

1. Review count per user.
2. Burst rate: the maximum number of reviews by a user in any 24 hour window, and in any 7 day window.
3. Rating extremeness ratio: share of a user's ratings that are 1 or 5.
4. Rating deviation: mean absolute difference between the user's rating and the item's average rating, where the item average excludes that user's own rating.
5. Sentiment rating mismatch: sentiment score from a lexicon model (VADER) or a small sentiment model, compared against the star rating.
6. Review length, and near duplicate score: cosine similarity of a review against the same user's other reviews and against other users' reviews on the same item.

Outputs. A data card in the report (sources, filters, counts, density), a cleaned data loading script, and a short limitations paragraph.

Risks. Large downloads and memory use. Mitigate by working on a sampled subset of items first, and by using Colab or Kaggle with sparse matrices.

Plan 2. Shilling Attack Simulation

Goal. Produce a controlled before versus after experiment that proves filtering helps, since real data has no ground truth for fake ratings.

Attack types to implement.

1. Random attack: fake users rate a chosen target item with the maximum rating (push) or minimum rating (nuke), plus filler items with ratings drawn from a normal distribution around the global mean and standard deviation.
2. Average attack: the same, but filler ratings are drawn around each filler item's own average rating, which makes profiles look more natural.
3. Bandwagon attack: fake users give the maximum rating to a few very popular items, then fill with random ratings, so they look similar to many real users.
4. Nuke variants of each, targeting a competitor item with the minimum rating.

Experimental design.

1. Choose 50 target items spread across popularity bins (rare, medium, popular).
2. Attack size: number of fake users as 1%, 3%, 5% and 10% of real users.
3. Filler size: a small number of filler items per fake user (for example 20, 50, 100).
4. Give each injected user a timestamp pattern that is bursty, and attach review text sampled from the held out computer generated class of the labeled dataset. Never use reviews the classifier was trained on, or detection will look unrealistically good.
5. Run each setting with five random seeds.

Metrics.

1. Prediction shift: average change in the predicted rating of the target item for genuine users after the attack.
2. Hit ratio at N: fraction of genuine users who see the target item in their top N.
3. RMSE and ranking metrics on held out genuine ratings only.

4. Detector precision, recall and F1 on the injected profiles.

Conditions to compare. No attack, attack with no filter, attack with hard filter, attack with soft weights (Plan 3), and the oracle that removes exactly the injected users (upper bound).

Figures. Prediction shift versus attack size for each condition; hit ratio versus attack size; detector recall by attack type.

Pitfalls. If attacked items are also in the test set, evaluate on genuine users and genuine ratings only. Keep the injection code and parameters in one config file so every run is reproducible.

Plan 3. Soft Weighting Instead of Hard Deletion

Goal. Use the classifier's fake probability as a confidence weight inside the factorization objective, rather than only deleting reviews.

Math. Let p be the fake probability of a rating and $w = (1 - p)$ raised to a sharpness power γ . The weighted objective over observed ratings is the sum of w times the squared error between the rating and the dot product of user and item factors, plus regularization on the factors.

For weighted ALS, fixing item factors gives a closed form per user:

```
# Solve one user's factors; Q_u holds factors of items this user rated
A = Q_u.T @ (w_u[:, None] * Q_u) + lam * n_u * np.eye(k)
b = Q_u.T @ (w_u * r_u)
p_u = np.linalg.solve(A, b)
```

The item step is symmetric. Scaling the regularizer by the user's rating count n_u is the weighted lambda regularization variant of ALS, which is more stable on uneven data. For weighted FunkSVD with gradient descent, multiply each rating's gradient step by its weight.

Variants to compare.

1. No filter (all weights equal to one).
2. Hard filter with thresholds 0.3, 0.5 and 0.7 on p .
3. Soft weights with gamma values 1, 2 and 4.
4. Weights from the fused score of text and behavior (Plan 5).

Why this stands out. It connects the detector to the linear algebra directly and gives you a tunable method, not just a preprocessing step. It also lets you discuss the tradeoff: hard removal throws away genuine reviews that were misclassified, while soft weights lose less information.

Important checks.

1. Calibrate the classifier probabilities first (Plan 5), since weights are only meaningful if p behaves like a probability.
2. Measure the cost on clean data with no attack. A good method should not hurt when there is nothing to remove.
3. Tune gamma and thresholds on a validation split, never on the test split.

Related idea for explanation. Implicit feedback ALS uses confidence weights too, but with a different meaning (confidence in a preference, not trust in a rating). Mention the difference so reviewers do not confuse them.

Plan 4. ALS From Graph and Graph Analytics

Plan 4. ALS From Scratch and Spectral Analysis

Goal. Show real computational mathematics, not only library calls.

Steps.

1. Write the derivation: set the gradient of the regularized least squares objective to zero for one user, obtain the normal equations, and show the matrix is symmetric positive definite when λ is positive. State the cost per user as order n_u times k^2 plus k^3 .
2. Implement ALS in NumPy with a sparse CSR rating matrix. Use mean centering plus user and item biases (global mean, user bias, item bias) before factorizing.
3. Track the objective after every half step. Because each half step is an exact minimization over its block, the objective can never increase. Plot it and state this property in the report.
4. Validate against a library implementation (Surprise SVD for FunkSVD, a Spark ALS run, or the implicit package) on the same split. Results should be close.
5. Rank selection: compute the leading singular values of the mean centered matrix with scipy sparse svds, plot the spectrum, and compare against validation RMSE for k in 5, 10, 20, 50 and 100.
6. Regularization study: sweep λ , plot validation RMSE, and discuss how λ controls the conditioning of the normal equations.
7. Truncated SVD note: explain that the Eckart Young theorem says truncated SVD is the best rank k approximation of a fully observed matrix, and why that does not directly apply here because most entries are missing. This is exactly why FunkSVD and ALS minimize error over observed entries only.
8. Spectrum under attack: plot the top singular values of the clean matrix versus the attacked matrix. Coordinated fake profiles tend to add low rank structure, which can show up as extra large singular values. If you see it, it makes a strong figure.

Outputs. A derivation page, clean code, convergence plot, spectrum plot, rank and λ sweeps, validation against the library.

Plan 5. A Stronger Detector

Goal. Move from four separate classifiers to a justified, calibrated detector.

Stage A, baselines. Logistic regression, Naive Bayes, linear SVM, decision tree, random forest and fine tuned DistilBERT. Use stratified five fold cross validation, and report accuracy, precision, recall, F1 and PR AUC with mean and standard deviation across folds.

Stage B, features. Compare TF IDF alone, embeddings alone, and TF IDF plus embeddings. Keep the vectorizer fitted inside each training fold only, otherwise you leak information.

Stage C, fusion. Behavior features cannot be learned from the text only labeled dataset. Two honest options:

1. Rule or score fusion: combine the text probability and a behavior anomaly score, such as the larger of the two or a weighted average, with weights tuned on the injected attack data from Plan 2.
2. Train a gradient boosting model (LightGBM or XGBoost) on behavior features plus text probabilities using the injected attacks as labels, and test it on attack types and parameters it never saw. Use out of fold text probabilities as inputs, never in sample ones.

Add an unsupervised anomaly detector (Isolation Forest) on behavior features to flag unusual accounts without labels, and compare it with the supervised fusion.

Stage D, calibration and thresholds. Plot a reliability curve, report the Brier score, and calibrate with Platt

scaling or isotonic regression. Sweep the decision threshold and plot precision against recall. Pick the threshold by how much genuine data you are willing to lose, and justify it.

Outputs. One comparison table, one PR curve figure, a calibration plot, and a written justification of the final detector.

Plan 6. Robustness Tests

Goal. Test the claim in your proposal that AI generated fakes are a growing threat.

Experiments.

1. Generate fresh fake reviews with a current language model across several electronics products and rating levels, using varied prompts (short, long, casual, formal). Use these only for evaluation, never for training, and measure recall.
2. Paraphrase attack: take detected fakes, lightly rewrite them, add typos, shorten them, and remove typical giveaway words. Plot recall against the strength of the perturbation.
3. Domain shift: if the labeled data has product categories, train on some categories and test on the held out ones. Report the drop.
4. Genuine review stress test: check the false positive rate on very short genuine reviews and on reviews from non native writers, since those are often wrongly flagged.

Outputs. A robustness table and a limitations section that says plainly where the detector fails. Honest failure analysis impresses reviewers more than inflated numbers.

Plan 7. Explainability

Goal. Make each flagged review justifiable.

Steps.

1. Global: SHAP values for the tree model and the largest coefficients for linear models. Show a bar chart of the most influential features.
2. Local for text: LIME on the TF IDF models, and token attribution (integrated gradients through Captum, or attention based views) for DistilBERT. Show five flagged reviews with highlighted words.
3. Reason codes: attach a short reason to each flagged account, such as "twelve reviews in 24 hours" or "text sounds negative, rating is five stars".
4. Case studies: pick three accounts and walk through what the detector saw, what was removed, and how that changed one product's rating.

Outputs. Figures for the report, and a reasons column in the flagged reviews table that the demo can display.

Plan 8. Evaluation Rigor

Goal. Make every comparison statistically credible.

Protocol.

1. Split: a temporal split (train on older ratings, test on newest) or per user leave last out. Fix the protocol and describe it before running anything.
2. Leakage control: fit the detector and any feature statistics on training data only.
3. Metrics. Accuracy: RMSE and MAE. Ranking: Precision, Recall, NDCG and MAP at 10, treating ratings of

4 or above as relevant, with a clearly stated candidate protocol (rank all unseen items, or sample negatives). Beyond accuracy: catalog coverage, novelty (average negative log popularity of recommended items), diversity (average pairwise distance between recommended items using item embeddings), and a popularity concentration measure such as the Gini index.

4. Tuning: grid search for k , λ , learning rate and iterations on a validation split.
5. Statistics: run five seeds, report means with bootstrap confidence intervals over users, and use a paired Wilcoxon test for SVD versus ALS and for filtered versus unfiltered.
6. Slices: cold start users (fewer than 10 ratings), cold start items, and heavy versus light users.

Output template. One master results table with rows for each model and filter condition, columns for each metric, and confidence intervals. Add a short paragraph that states the winner and whether the gap is statistically significant.

Plan 9. Product Trust Score

Goal. Give a simple, visual outcome of filtering that anyone can understand.

Steps.

1. Compute each product's raw average rating and its trust adjusted average, weighted by the weights from Plan 3. Apply Bayesian smoothing so products with very few ratings are not extreme.
2. Rank products by raw and by adjusted rating. Report Kendall tau between the two rankings.
3. List the products whose rank changes the most, and show how many flagged reviews each had.
4. Using the attack simulation, show that attacked items are inflated in the raw ranking and return to a realistic position in the adjusted ranking.

Outputs. A top movers table, a before versus after scatter plot, and a one line headline number for the abstract.

Plan 10. Graph Based Detection

Goal. Catch coordinated fake accounts that look normal individually.

Steps.

1. Build the user to product bipartite graph from the cleaned data.
2. Project it to a user to user graph where the edge weight grows with the number of shared products, similar ratings on them, and closeness in time.
3. Keep edges above a threshold, run community detection (Louvain in networkx), and score each community by size, density, and rating similarity.
4. Feed the community score into the fused anomaly score from Plan 5.
5. Evaluate on injected attack groups: recall of group members, and how many genuine users are wrongly placed in suspicious communities.

Why it matters. Text and per user behavior can miss paid groups, but groups leave a structural footprint. This is a good extension and also connects to the graph and matrix topics in your minor.

Risk. User to user projections can become huge. Restrict to items with a moderate number of reviewers, or sample, and document the choice.

Plan 11. Reusing Embeddings for Cold Start

Goal. Get a second use out of the DistilBERT embeddings you already extract.

Steps.

1. Build an item vector as the mean of the embeddings of its cleaned reviews (after filtering or weighted by trust).
2. For items with very few training ratings, estimate their latent factors from the latent factors of the nearest items in embedding space, or learn a small regression from embedding to latent factor using well rated items.
3. Blend: use the collaborative factors when an item has enough ratings and the embedding estimate when it does not.
4. Evaluate only on items with three or fewer training ratings, and compare against the plain model.

Outputs. A cold start results table and a note that filtering improves the embeddings themselves because fake text no longer pollutes them.

Plan 12. Demo and Packaging

Goal. Make the project easy to present and easy to rerun.

Demo app (Streamlit or Gradio), three tabs.

1. Review checker: paste a review and rating, see the fake probability and the top reasons.
2. Recommender comparison: pick a user, see top 10 items under no filter, hard filter and soft weights, with rank changes highlighted.
3. Attack lab: sliders for attack size and target item, showing the target's predicted rating and rank before and after, with and without defense.

Repository structure.

1. data (not committed; a README explains how to download it)
2. src with modules for loading, features, detector, recommenders, attacks and evaluation
3. configs with a single config file for seeds and parameters
4. notebooks for figures and exploration
5. results for saved tables and plots
6. app for the demo

Packaging checklist.

1. A README with the project summary, architecture figure, setup steps, how to reproduce each table, and known limitations.
2. A requirements.txt with pinned versions, and fixed random seeds everywhere.
3. Large model files kept out of git (store the DistilBERT checkpoint on Hugging Face or Drive and link it).
4. A short test file for the loaders, the ALS update, and the attack generator.
5. A run script that regenerates all results from scratch.

Plan 13. Report and Presentation Updates**Changes to the proposal.**

1. Update the objectives to include: attack simulation, soft weighting, and statistical comparison.
2. Add to the literature review: shilling attack models and robust collaborative filtering, weighted matrix

factorization, and calibration of classifiers.

3. Rewrite the research gap as one sentence you can defend: prior work either filters fake reviews or builds factorization models, and this project connects the two and measures the effect under controlled attacks.
4. Add a threats to validity section: dataset mismatch, synthetic attacks, domain shift, and label noise.

Claims you can safely make if the plans above are completed.

1. Filtering at a given threshold reduces prediction shift and hit ratio under push attacks by a measured amount.
2. Soft weighting matches or beats hard filtering while losing less genuine data (verify before claiming).
3. Whether SVD or ALS is better on this data, with confidence intervals.

Claims to avoid. Do not claim the detector works on all fake reviews, or that results on injected attacks equal real world performance. Say what was measured and on what.

Quick Reference: Effort and Impact

1. Plan 2 attack simulation: high impact, medium effort.
2. Plan 3 soft weighting: high impact, medium effort.
3. Plan 1 data fix: high impact, medium effort, do first.
4. Plan 5 stronger detector: high impact, medium effort.
5. Plan 4 ALS from scratch: medium impact, medium effort, strong for the minor.
6. Plan 8 evaluation rigor: high impact, low to medium effort.
7. Plan 12 demo: medium impact, medium effort.
8. Plans 7, 9: medium impact, low effort.
9. Plans 6, 10, 11: nice extras, medium to high effort.

